Household Electricity Consumption Analysis

Python Programming — Individual Case Study (Internal Assessment)

Name	Manjari Majumdar
Roll Number	RA2411005010075
Programme	B.Tech Electrical and Electronics Engineering
Institution	SRM Institute of Science and Technology, Chennai
Course	Python Programming
Submission date	30 October 2026
Files	RA2411005010075_ManjariMajumdar_CaseStudy.ipynb RA2411005010075_ManjariMajumdar_Dataset.csv

1. Problem Statement

Households usually see only one number every month: the total electricity bill. They do not know which appliances consume the most energy, which days are unusually heavy, or how slab-based tariffs push the bill up once consumption crosses a limit. Without this breakdown it is difficult to decide where to save energy. This case study uses fundamental Python programming to analyse 30 days of appliance-wise electricity consumption for a household and convert the raw readings into useful information.

2. Objectives

- Read and organise daily appliance-wise consumption data (kWh) from a CSV file.
- Find the total and average consumption of each appliance and its percentage share.
- Identify high-usage days that exceed a chosen threshold (14 kWh/day).
- Compare average weekday and weekend consumption.
- Estimate the monthly bill using a slab-based tariff and test a simple energy-saving scenario.
- Visualise the results and draw conclusions for reducing energy use.

3. Dataset Description

The dataset was **simulated by the student** to represent a typical 2BHK household in Chennai for 1–30 September 2026. Realistic ranges were used for each appliance, with higher AC usage on weekends and during a hot spell in the second week (9–15 September), and washing machine use on Wednesdays and weekends. It is submitted separately as *RA2411005010075_ManjariMajumdar_Dataset.csv*.

Column	Type	Description
Date	Text	Date of reading (YYYY-MM-DD)
Day	Text	Day of the week
AC	Float (kWh)	Air conditioner energy for the day
Refrigerator	Float (kWh)	Refrigerator energy for the day
Water_Heater	Float (kWh)	Water heater (geyser) energy
Lights_Fans	Float (kWh)	Combined lighting and ceiling fans
Washing_Machine	Float (kWh)	Washing machine energy (0 on non-wash days)
Laptop_TV	Float (kWh)	Laptop, television and small electronics

Size: 30 rows × 8 columns. No missing values.

4. Algorithm / Step-by-step Methodology

1. Start. Define constants: appliance list, a **set** of weekend day names, the tariff as a **list of tuples** (slab upper limit, rate), and the high-usage threshold.
2. **load_data()**: open the CSV with the *csv* module; for each row create a **dictionary**, convert readings to float, and append it to a **list**.
3. **appliance_totals()**: initialise a dictionary of zeros and use nested **for loops** to add each day's reading to its appliance total.

4. **daily_totals()**: for each day, sum all appliance readings and store the result in a list.
5. **find_high_usage_days()**: compare each daily total with the threshold using an **if** statement; store matching days as (date, day, kWh) tuples.
6. **weekday_vs_weekend()**: use **if-else** with set membership to accumulate weekday and weekend sums and counts, then compute averages.
7. **calculate_bill()**: loop over the tariff slabs; for each slab compute units falling in it, multiply by the rate, and accumulate the bill.
8. Print the results in formatted tables and plot four graphs with *matplotlib*.
9. Interpret the outputs and write observations and conclusions. Stop.

5. Python Implementation

The complete code with execution outputs is in the submitted notebook. The six user-defined functions that form the core of the program are reproduced below.

```
def load_data(filename):
    """Read the CSV file and return a list of dictionaries, one per day."""
    records = []
    with open(filename, "r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            day_record = {"Date": row["Date"], "Day": row["Day"]}
            for appliance in APPLIANCES:
                day_record[appliance] = float(row[appliance])   # convert text to number
            records.append(day_record)
    return records


def appliance_totals(records):
    """Return a dictionary {appliance: total kWh for the month}."""
    totals = {}
    for appliance in APPLIANCES:
        totals[appliance] = 0.0
    for day in records:
        for appliance in APPLIANCES:
            totals[appliance] += day[appliance]
    return totals


def daily_totals(records):
    """Return a list with the total kWh used on each day."""
    totals = []
    for day in records:
        day_sum = 0.0
        for appliance in APPLIANCES:
            day_sum += day[appliance]
        totals.append(round(day_sum, 2))
    return totals


def find_high_usage_days(records, totals, threshold):
    """Return a list of (date, day, kWh) tuples for days above the threshold."""
    high_days = []
    for i in range(len(records)):
        if totals[i] > threshold:
            high_days.append((records[i]["Date"], records[i]["Day"], totals[i]))
    return high_days


def weekday_vs_weekend(records, totals):
    """Return the average daily kWh on weekdays and on weekends."""
    weekday_sum, weekday_count = 0.0, 0
    weekend_sum, weekend_count = 0.0, 0
    for i in range(len(records)):
        if records[i]["Day"] in WEEKEND_DAYS:
```

```
            weekend_sum += totals[i]
            weekend_count += 1
        else:
            weekday_sum += totals[i]
            weekday_count += 1
    return {"Weekday": weekday_sum / weekday_count, "Weekend": weekend_sum / weekend_count}


def calculate_bill(units):
    """Apply the slab tariff and return (total bill, slab-wise breakdown list)."""
    bill = 0.0
    breakdown = []
    lower = 0
    for upper, rate in TARIFF_SLABS:
        if units > lower:
            units_in_slab = min(units, upper) - lower
            cost = units_in_slab * rate
            bill += cost
            breakdown.append((lower, upper, round(units_in_slab, 2), rate, round(cost, 2)))
        lower = upper
    return round(bill, 2), breakdown
```

6. Results and Visualisations

6.1 Appliance-wise consumption

Appliance	Total (kWh)	Average (kWh/day)	Share (%)
AC	167.28	5.58	44.3
Refrigerator	43.07	1.44	11.4
Water_Heater	37.08	1.24	9.8
Lights_Fans	76.72	2.56	20.3
Washing_Machine	20.20	0.67	5.3
Laptop_TV	33.31	1.11	8.8
TOTAL	377.66	12.59	100.0

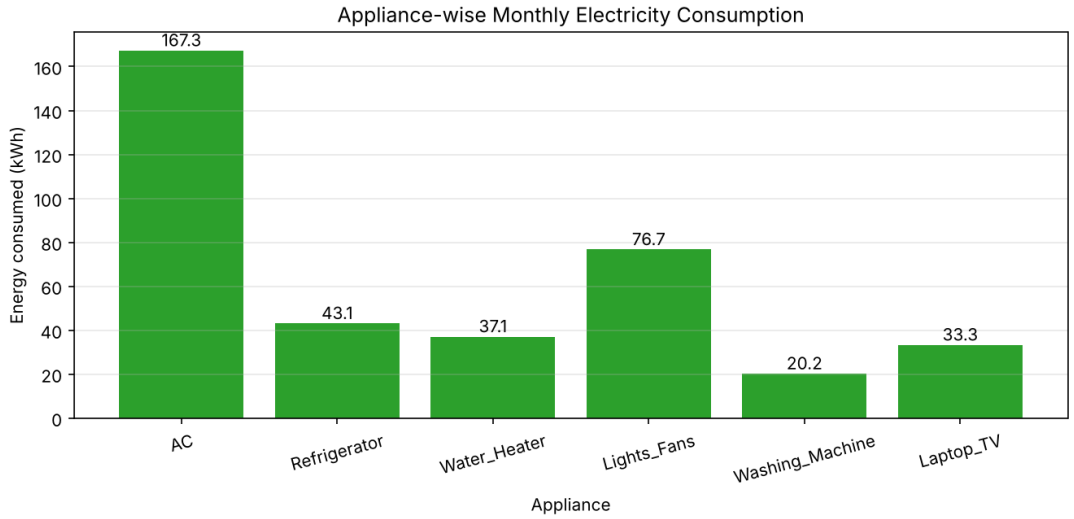


Figure 1: Appliance-wise monthly consumption (kWh)

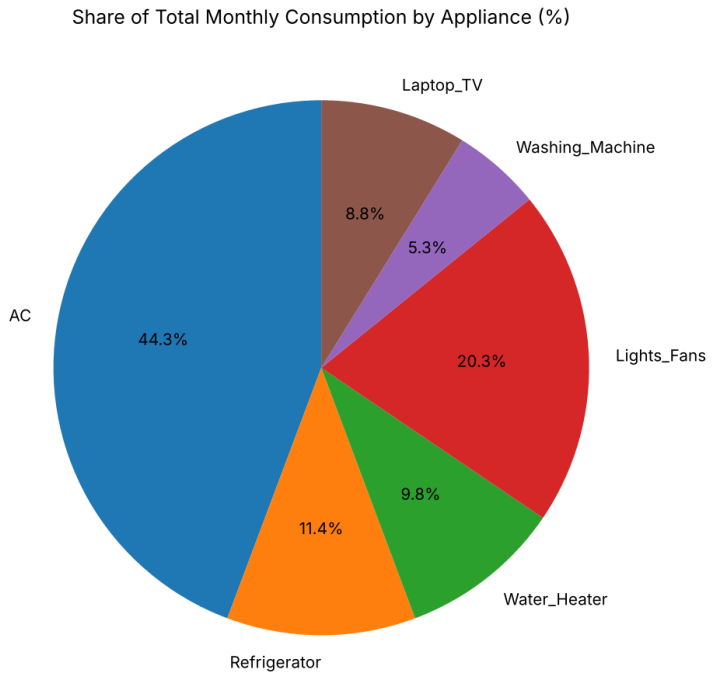


Figure 2: Percentage share of each appliance in total consumption

6.2 Daily consumption and high-usage days

Maximum: 16.92 kWh (13 Sep, Sunday). Minimum: 9.87 kWh (24 Sep). Average: 12.59 kWh/day. Nine days exceeded the 14 kWh threshold.

Date	Day	Consumption (kWh)
2026-09-05	Saturday	15.54
2026-09-06	Sunday	15.36
2026-09-09	Wednesday	14.19
2026-09-12	Saturday	16.51
2026-09-13	Sunday	16.92
2026-09-19	Saturday	15.9
2026-09-20	Sunday	15.01
2026-09-26	Saturday	15.79
2026-09-27	Sunday	14.97

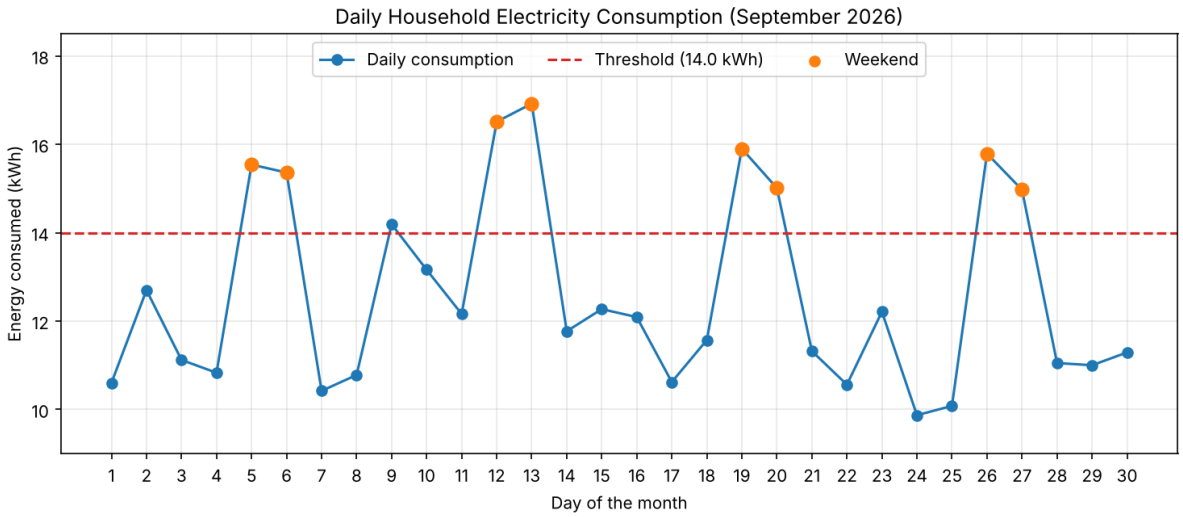


Figure 3: Daily consumption with 14 kWh threshold; weekends marked in orange

6.3 Weekday vs weekend

Category	Average consumption (kWh/day)
Weekday (22 days)	11.44
Weekend (8 days)	15.75
Difference	+37.7 % on weekends

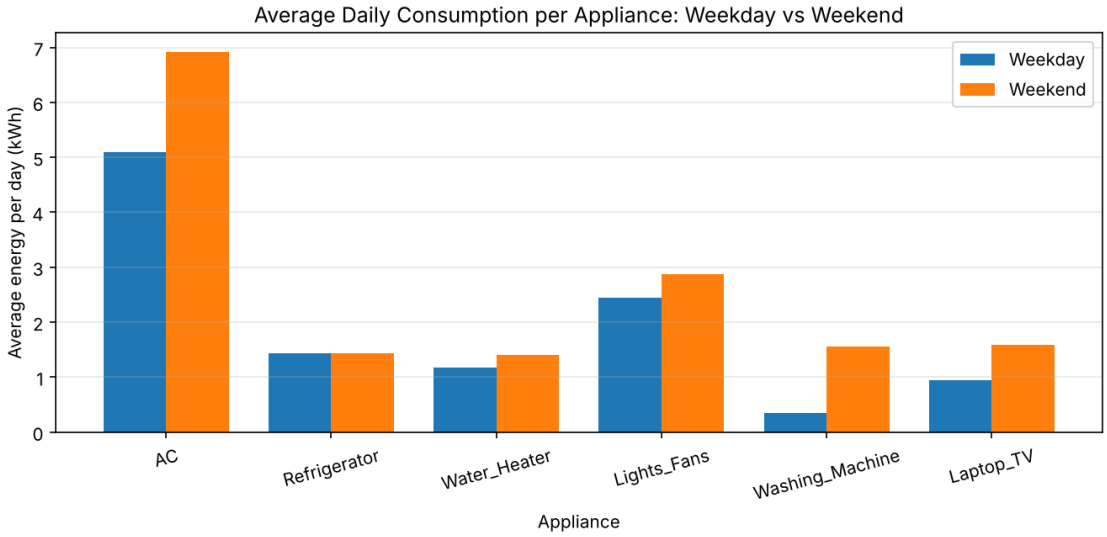


Figure 4: Average daily consumption per appliance, weekday vs weekend

6.4 Monthly bill estimate (slab tariff)

Slab (kWh)	Units	Rate (Rs/kWh)	Cost (Rs)
0 – 100	100.00	0.00	0.00
101 – 200	100.00	2.35	235.00
201 – 400	177.66	4.70	835.00
Total	377.66		1070.00

What-if scenario: reducing AC usage by 20% lowers consumption to 344.20 kWh and the bill to Rs 912.76, a saving of Rs 157.24 per month. (Tariff values are illustrative, modelled on the slab structure of Tamil Nadu domestic tariffs.)

7. Observations

- **The air conditioner dominates consumption.** AC used 167.28 kWh, which is 44.3% of the month's 377.66 kWh (bar and pie charts). Lights and fans come second at 20.3%. Together these two account for almost two-thirds of the bill, so they are the main targets for saving energy.
- **Weekends are much heavier than weekdays.** Average use was 15.75 kWh/day on weekends against 11.44 kWh/day on weekdays, which is 37.7% higher. The grouped bar chart shows the biggest weekend jump comes from the AC (about 6.9 vs 5.1 kWh/day) and the washing machine, because people are at home for longer.
- **High-usage days follow a clear pattern.** 9 of 30 days crossed the 14 kWh threshold. Eight of them are Saturdays and Sundays, and the only weekday on the list (9 September) falls in the hot spell of the second week. The highest day was Sunday, 13 September (16.92 kWh), where a weekend and hot weather combined.
- **Weather affects AC load.** During the hot spell (9–15 September), weekday AC use averaged about 6.2 kWh/day, compared with about 4.8 kWh/day on other weekdays, an increase of roughly 30%. This shows up as the raised section in the middle of the daily trend line.
- **Base loads are steady.** The refrigerator stayed between 1.27 and 1.64 kWh every day because it runs continuously, so it cannot be "switched off" to save energy. Savings must come from controllable loads like the AC.
- **The slab tariff makes savings worth more than they appear.** The household reached the third slab (201–400 kWh at Rs 4.70/kWh), giving an estimated bill of Rs 1070.00. Cutting AC use by just 20% (for example by raising the set temperature to 26 °C) reduces consumption to 344.20 kWh and saves about Rs 157 per month (≈15%), because every unit saved comes off the most expensive slab.

8. Conclusion

The program successfully turned 30 days of raw readings into a clear picture of household energy use using only basic Python: lists and dictionaries to store data, loops and conditionals to analyse it, a set to classify days, tuples to represent tariff slabs, and six user-defined functions. The analysis shows that the AC is the single largest consumer, weekends and hot days drive the peaks, and modest changes to AC use would give a noticeable saving because of the slab tariff. The same program could be reused with real smart-meter data to monitor consumption every month.

9. Acknowledgement

The dataset was simulated by the student; no external dataset was used. Tariff slabs are illustrative. Libraries used: Python *csv* (built-in) and *matplotlib*.